

CLAUDE CODE · MANUEL

Le manuel que tu finis vraiment.

52 pages. Gratuit. Pour les devs et les knowledge workers. Hooks, sub-agents, skills, développement plan-first, et la boucle d'audit qui empêche tout ça de pourrir avec le temps.

Pourquoi ce livre existe

Si tu utilises Claude Code depuis plus de deux semaines, tu as sans doute remarqué un motif : les 10 premières minutes de chaque session sont consacrées à ré-expliquer ta stack, tes conventions, ce que tu as corrigé hier. Tu corriges les mêmes dérives encore et encore. Tu découvres que Claude a écrit une commande destructrice dans ton `.env`. Tu cesses de lui faire confiance pour quoi que ce soit de sérieux.

Rien de tout cela n'est la faute de Claude. C'est un artefact d'un LLM lancé sans contexte projet, sans garde-fous, sans mémoire. Ce manuel corrige ça — pas avec un framework ni une abstraction lourde, mais avec un petit ensemble de fichiers qui vivent dans ton repo et transforment Claude Code en collaborateur reproductible et opinionné.

À la fin du livre, tu auras :

- Un contexte projet permanent (un `CLAUDE.md` chargé à chaque session)
- Des garde-fous défensifs qui bloquent les commandes destructrices
- Des sub-agents spécialisés pour l'audit code, la sécu, et ton domaine métier
- Un système de skills pour les connaissances métier qui survivent aux re-clones
- Un workflow plan-first avec un « staff engineer » qui relit avant que tu codes
- Une boucle d'audit qui détecte la dérive avant toi

L'ensemble prend ~30 minutes à amorcer et quelques heures à affiner. Le gain composé se mesure en jours économisés par trimestre.

Ceci est la V2 du manuel. Nouveau : chapitres complets sur les skills, le workflow plan-first inspiré du créateur de Claude Code Boris Cherny, la boucle d'audit, les worktrees parallèles, et la matrice mémoire à 4 lieux. Les deux études de cas en annexe (un dev solo et une knowledge worker) montrent le setup en vrai.

POUR QUI

Ce livre est pour toi si...

Oui, c'est pour toi

- **Tu es développeur** (toute stack, tout niveau) et tu utilises Claude Code au quotidien — ou veux le faire.
- **Tu es un knowledge worker** — rédacteur, chercheur, analyste — qui utilise Claude Code pour écrire, faire de la recherche, générer du contenu, et tu veux un setup stable et reproductible plutôt que de recommencer à chaque session.
- **Tu es solo founder ou lead d'une petite équipe** et tu veux multiplier ton output sans embaucher.
- **Tu fais partie d'une équipe qui veut une baseline partagée et opinionnée** pour que tout le monde configure Claude pareil — et que le setup lui-même soit reviewable en PR.
- **Tu n'es pas natif anglais** — tout le livre est en français avec la même profondeur, aucun raccourci.

Ce livre n'est PAS pour toi (encore) si...

- **Tu n'as jamais ouvert un terminal.** Commence par la doc officielle Anthropic — `code.claude.com/docs`. Reviens quand tu as installé Claude Code et eu une 1ère session.
- **Tu veux 500 pages de référence encyclopédique.** Ici c'est 52 pages, lisibles en 2 heures. Si tu veux le format encyclopédie avec 181 templates de skills et 271 questions de quiz, le Ultimate Guide de Florian Bruniaux est excellent.

- **Tu attends que ce livre écrive du code à *ta place*.** Ce livre parle de *configurer Claude Code*, pas de te remplacer par lui. Tu écriras toujours le code, les plans, les décisions. Claude sera juste plus utile, plus rapide, plus fiable.

Ce que tu auras à la fin

- Un repo où chaque nouvelle session Claude Code charge ton contexte automatiquement, refuse les opérations destructrices, et se souvient de ce que tu lui as appris la semaine dernière.
- Une petite boîte à outils d'assistants spécialisés (sub-agents) à qui tu fais assez confiance pour déléguer.
- Une discipline — planifier avant de coder, capter les leçons après — qui se compose dans le temps.

Note sur le vocabulaire : ce livre utilise quelques termes spécifiques (hook, sub-agent, frontmatter, skill, MCP). Chacun est défini à sa première apparition et à nouveau dans le glossaire (Annexe F). Pas besoin de les apprendre avant ; continue.

Ce livre

Trois parcours de lecture

Si tu es...	Lis dans cet ordre	Passe
Novice sur Claude Code	Avant-propos → Partie I → Partie II → Annexe G (checklist)	Parties III, IV, V pour l'instant
À l'aise mais ad-hoc	Partie I (survol) → Partie II → Partie III en entier	—
Power user qui migre de V1	What's new (avant-propos) → Chap. 06 → 07 → 08 → 10 → 11 → étude de cas	—

Conventions typographiques

- code dans le texte — chemins de fichiers, commandes, noms de variables
- Gras** — concepts introduits pour la première fois
- Italique* — citations de l'interface utilisateur ou premier usage d'un terme externe

Badges de difficulté

Chaque chapitre porte un badge :

- DÉBUTANT** Les 30 premières minutes de setup ; aucun pré-requis Claude Code.
- INTERMÉDIAIRE** Construit sur les bases ; suppose qu'au moins un hook et un agent sont configurés.
- AVANCÉ** Patterns de workflow et méta-outillage. À lire après une semaine d'usage du setup.

Encadrés

ASTUCE	Une recommandation pratique applicable immédiatement.
ATTENTION	Un piège facile à manquer mais coûteux à réparer.
NE FAIS PAS ÇA	Un pattern qui a l'air raisonnable mais casse activement le setup.
CONTEXTE	« Pourquoi » — sautable si tu nous fais confiance.

Où trouver quoi

PARTIE I — FONDATIONS	9
00 — Choisir son interface	10
01 — Philosophie & ROI	12

PARTIE II — SETUP DE BASE	17
02 — Le setup minimum viable (30 min)	18
03 — Hooks	23
PARTIE III — CONFIGURATION SPÉCIALISÉE	30
04 — Slash commands	31
05 — Sub-agents	34
06 — Skills (la bonne méthode)	40
PARTIE IV — WORKFLOW AVANCÉ	46
07 — Développement plan-first	47
08 — La boucle d'audit	50
09 — Mode Coach	53
PARTIE V — POUR ALLER PLUS LOIN	56
10 — Worktrees parallèles	57
11 — La matrice mémoire	59
12 — MCP — connecter à l'extérieur	62
13 — Maintenance long-terme	65
ANNEXES	67
A. CLAUDE.md universel	68
B. .claude/settings.json complet	69
C. Les six hooks (condensés)	70
D. Quatre templates d'agents	71
E. Six slash commands	72
F. Glossaire	73
G. Quick start 30 minutes	74
H. Étude de cas (anonymisée)	75

PARTIE I

Fondations.

Pourquoi un Claude Code configuré bat un improvisé. Les 3 principes. Ce que le setup t'apporte, et ce qu'il n'apporte pas.

Choisir son interface

Avant de configurer quoi que ce soit, sache où tu vas l'installer. Toutes les surfaces Claude Code supportent les mêmes mécanismes — mais Claude Code reste distinct du produit Claude.ai (chat).

Les 6 surfaces officielles Claude Code

Toutes lisent la même config `.claude/` dans ton repo. Tu peux switcher entre elles sans rien re-configurer.

Surface	Particularité
Terminal CLI	Le plus complet. Commande <code>claude</code> dans n'importe quel dossier. Idéal pour scripting, pipes, CI.
VS Code extension	Inline diffs, @-mentions, plan review. Marche aussi dans Cursor (extension officielle Anthropic, pas <code>.cursorrules</code>).
JetBrains plugin	IntelliJ, PyCharm, WebStorm, etc. Marketplace JetBrains officielle.
Desktop app	App standalone macOS/Windows. Diff visuel, sessions parallèles, tâches planifiées, sessions cloud.
Web	<code>claude.ai/code</code> dans le navigateur. Idéal pour tâches longues, repos pas en local, mobile.
iOS app	Claude app pour iPhone. Continuer une session depuis le téléphone.

LE MOTEUR EST PARTAGÉ

Chaque surface se connecte au même moteur Claude Code. Ton `CLAUDE.md`, tes hooks, tes sub-agents, ton `.mcp.json` — tout marche partout. Tu démarres une tâche dans le terminal, tu la reprends sur ton tel.

Recommandation par profil

Profil	Recommandation
Solo dev qui code dans un IDE	VS Code ou JetBrains extension pour le quotidien, CLI dans le terminal pour les workflows scriptés. Setup complet possible.
Équipe	N'importe quelle surface, avec un setup <code>.claude/</code> versionné dans git. Tout le monde hérite de la config.
Mobile / hors du bureau	Web sur tel ou iOS app . Lance une tâche longue, reprends-la sur l'ordi.
Utilisateur Cursor	Installer l'extension Claude Code officielle dans Cursor (<code>cursor:extension/anthropic.claude-code</code>). Toute la config <code>.claude/</code> marche.

CLAUDE.AI (CHAT) ≠ CLAUDE CODE

Claude.ai (le produit chat) supporte les « Projects » avec instructions custom (équivalent léger à `CLAUDE.md`), mais **pas** hooks, sub-agents, ou MCP serveur. Ce manuel concerne Claude Code (les 6 surfaces ci-dessus). Pour le chat seul, garde uniquement les chapitres 1-2 (principes + `CLAUDE.md`).

Philosophie & ROI

La différence entre quelqu'un qui utilise Claude et quelqu'un qui en tire 10× plus, ce n'est pas le prompt. C'est ce qui se passe *autour* du prompt.

Le problème du « vibe-coding »

Quand tu utilises Claude sans configuration, voici ce qui se passe à chaque session :

1. Tu réexpliques ta stack (« on est en Next.js 16, attention au breaking de revalidate... »).
2. Tu réexpliques tes conventions (« camelCase pour les fonctions, kebab-case pour les fichiers... »).
3. Claude part dans une direction, tu corriges (« non, ici on utilise pas `alert()`, c'est un toast »).
4. Claude lance une commande dangereuse, tu valides 4 pop-ups pour des trucs anodins.
5. Tu oublies de lancer `pnpm verify` avant un push, la CI crash.
6. Tu refais la même séquence demain.

Multiplie par 250 sessions par an. Tu réalises que tu as passé des semaines à faire du **contexte jetable**.

L'idée centrale

Tout ce que tu dis à Claude plus de 3 fois doit vivre dans un fichier. Tout ce que Claude fait plus de 3 fois sans intervention humaine doit être un hook.

Les 3 principes

PRINCIPE 1 — CAPTER LE CONTEXTE

Stack, conventions, gotchas, structure du projet → `CLAUDE.md`. Claude le lit à chaque session, automatiquement. Tu n'as plus à expliquer « on est en Next.js 16 ».

PRINCIPE 2 — DÉTERMINER L'AUTOMATISABLE

Auto-format après chaque édition, log des commandes, rappels en début de session → **hooks**. Ce sont des scripts (bash, Python) qui tournent sur des événements. Zéro token, zéro friction.

PRINCIPE 3 — SPÉCIALISER LES RÔLES

Audit sécu, validation de formules, scaffold de composant → **sub-agents**. Chaque sub-agent a son contexte, son modèle, ses outils. Tu lances un audit sécu sans polluer la session principale.

Le ratio temps investi / temps gagné

Élément du setup	Temps de mise en place	Gain par session	Break-even
CLAUDE.md de base	15 min	2-5 min	3 sessions
Permissions allow	10 min	1-3 pop-ups évités	1 session
Hook PreToolUse défensif	20 min	0 (mais sauve 1 catastrophe/an)	1 ^{re} catastrophe évitée
Hook PostToolUse format	15 min	30 secondes / write	3 sessions
5 sub-agents ciblés	2 h	10-30 min / audit	~10 audits
3 slash commands	45 min	2 min / commande	~20 invocations

Total setup : **une demi-journée**. Gain estimé : **15-30 min par jour de travail actif**. Sur 200 jours par an, ça fait 50-100 heures économisées.

Ce que ce setup ne fait PAS pour toi (nouveau en V2)

C'est tentant de lire le reste du livre comme « la config magique qui résout tout ». Elle ne le fait pas. Connaître les limites à l'avance évite la déception après.

- **Elle ne va pas rendre Claude plus rapide.** Le nombre de tokens augmente légèrement (CLAUDE.md, skills, prompts d'agents consomment tous du contexte). Le ROI est dans les corrections évitées, pas la génération plus rapide.
- **Elle ne va pas écrire tes tests pour toi.** Un agent `code-auditor` flagge les tests manquants ; il ne les écrit pas.
- **Elle ne remplace pas le jugement humain.** Plan-reviewer (Chapitre 07) est un 2e avis, pas un gate. C'est toi qui décides.
- **Elle ne va pas fixer ta codebase rétroactivement.** Un setup propre appliqué à 50k LOC en bordel laisse le bordel. Le setup se compose pour l'avenir.
- **Elle ne supprime pas le besoin de relire le code.** Si tu délègues tout aveuglément sans jamais lire le diff, tu as remplacé un problème par un pire.

MODÈLE MENTAL

Vois ça comme une *ceinture de sécurité et un GPS*, pas une voiture autonome. C'est toujours toi qui conduis. Tu cesses juste de te crasher dans les mêmes murs.

PARTIE II

Setup de base.

30 minutes pour une config fonctionnelle. Puis les 4 hooks qui rendent Claude fiable.

Le setup minimum viable

Objectif : transformer ton Claude en 30 minutes. Aucun pré-requis. Juste 3 fichiers à créer. À la fin, une nouvelle session dans ton repo chargera ton contexte automatiquement, refusera les opérations destructrices, et n'aura plus de pop-ups pour les commandes `git` / `ls` / `cat` habituelles.

Étape par étape

1. Installer Claude Code

Selon ton OS et ta surface :

```
BASH

# macOS, Linux, WSL – installation native (recommandée, auto-update)
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# macOS via Homebrew
brew install --cask claude-code

# Windows via WinGet
winget install Anthropic.ClaudeCode
```

Puis dans n'importe quel dossier de projet :

```
BASH

cd ton-projet
claude
```

Pour les IDE : Extensions → chercher « Claude Code » (VS Code, Cursor, JetBrains). Pour le navigateur : `claude.ai/code` . Pour l'app : télécharger depuis `claude.ai` . Toutes ces surfaces lisent les mêmes fichiers `.claude/` .

2. Créer `CLAUDE.md` à la racine du projet

C'est le brain de ton projet. Claude le charge automatiquement à chaque session. Template minimal :

MARKDOWN

<Nom du projet>

Description en 1 phrase de ce que fait le projet.

Stack technique

- Langage : <TypeScript 5 | Python 3.12 | Go 1.22 | ...>
- Framework : <Next.js 16 | FastAPI | Rails | ...>
- DB : <Postgres | SQLite | MongoDB | ...>
- Tests : <vitest | pytest | jest>
- Déploiement : <Vercel | Fly.io | Railway | ...>

Commandes utiles

- <commande dev> - lancer le serveur local
- <commande verify> - type-check avant push
- <commande test> - lancer la suite de tests

Conventions

- <Règle de nommage> (ex : camelCase JS, kebab-case fichiers)
- <Règle de structure> (ex : chaque page a son <actions.ts>)
- <Règle de comportement> (ex : jamais <alert()>, toujours toast)

Gotchas

- <Piège connu de la stack>
- <Piège interne au projet>

Workflow Git

- Branche cible : <main | master>
- Format de commit : <type(scope): description>

Règles impératives

- <Ne jamais> <X> sans confirmation
- <Toujours> <Y> avant <Z>

3. Créer .claude/settings.json

Le fichier qui contrôle les permissions et les hooks. Template minimal qui élimine 80 % des pop-ups :

JSON

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Read",
      "Grep",
      "Glob",
      "Bash(git status:*)",
      "Bash(git diff:*)",
      "Bash(git log:*)",
      "Bash(git show:*)",
      "Bash(git branch:*)",
      "Bash(ls:*)",
      "Bash(cat:*)",
      "Bash(find:*)",
      "Bash(head:*)",
      "Bash(tail:*)",
      "Bash(wc:*)"
    ],
    "deny": [
      "Bash(rm -rf*)",
      "Bash(sudo*)",
      "Bash(curl* | bash*)"
    ]
  }
}
```

Adapte la liste `allow` à ta stack (ajoute `"Bash(pnpm verify)"`, `"Bash(npm test:*)"`, etc.).

4. Ajouter `.claude/` au `.gitignore` (sélectivement)

Tu veux versionner les fichiers d'équipe mais pas les logs locaux :

GITIGNORE

```
# Claude Code – versionner agents/commands/hooks/settings.json,  
# mais pas les logs ni les overrides perso  
.claude/*  
!.claude/agents/  
!.claude/commands/  
!.claude/hooks/  
!.claude/settings.json  
.claude/logs/  
.claude/settings.local.json
```

5. Tester

Lance Claude Code. Pose une question : « *Lis CLAUDE.md et résume la stack en 3 lignes.* » S'il te répond correctement avec ce que tu as mis dans le fichier →  ça marche. Sinon : vérifie que le fichier est bien à la racine du projet, pas dans `src/` ou ailleurs.

INIT AUTO

Claude Code propose une commande `/init` qui scanne ton repo et propose une première version de `CLAUDE.md`. Lance-la, puis ajuste : retire ce qui est faux, ajoute tes règles spécifiques.

ANTI-PATTERN

Mettre 800 lignes dans `CLAUDE.md` le premier jour. Le fichier doit rester **stable et lisible**. Si tu te retrouves à scroller pour trouver une info, il est temps de splitter (Chapitre 13).

Ce que tu as gagné en 30 minutes

- Plus jamais besoin de réexpliquer la stack ou les conventions.
- Plus de pop-ups pour les commandes `git / ls / cat` classiques.
- Les commandes dangereuses (`rm -rf` , `sudo`) sont bloquées.
- Le repo est prêt pour les niveaux suivants.

Checkpoint — à quoi ressemble une session bien configurée (nouveau en V2)

Après avoir lancé `claude` dans un repo fraîchement configuré, tu devrais voir quelque chose comme :

TERMINAL

```
todo-api • session démarrée • Ven 22 Mai 09h22
```

```
Branch      : main  
Working tree : clean  
Derniers commits :  
  a3b1f0 feat(api): add DELETE /todos/:id  
  9d2e84 fix(db): handle missing connection string  
  1c7a55 chore: bump express to 4.21
```

Si tu vois ça, 3 choses marchent : le hook `SessionStart` tourne, ton `CLAUDE.md` est chargé silencieusement, et ta commande `git status` est autorisée par l'`allowlist`. Si quelque chose cloche, va voir *Debug d'un hook silencieux* au chapitre 03.

Hooks

Un hook est un script déclenché par un événement. Zéro token, zéro intervention de Claude, zéro friction. C'est la couche d'« automatisation déterministe » que Claude ne voit même pas.

Les 4 événements clés (parmi ~25)

Claude Code expose environ 25 événements. Ce manuel couvre les **4 qui gèrent 80 % des besoins**. Liste complète dans la doc officielle code.claude.com/docs/en/hooks.

Événement	Quand	Cas d'usage typiques
SessionStart	Au démarrage de Claude	Récap git, fetch silencieux, état du projet
PreToolUse	Avant chaque appel d'outil	Bloquer <code>rm -rf</code> , écritures dans <code>.env</code> , push <code>--force</code>
PostToolUse	Après chaque appel d'outil	Auto-format, activity log, nudges de la boucle d'audit
Stop	À la fin du tour de Claude	Notification, mode Coach, ping Slack/Discord

Hook #1 — PostToolUse : auto-format après chaque écriture

Crée `.claude/hooks/post-edit-format.sh` :

```
BASH

#!/usr/bin/env bash
# Auto-format des fichiers après Write/Edit.
# Adapte le formater à ta stack : prettier, black, gofmt, rustfmt...
set -e

INPUT="$(cat 2>/dev/null || echo '{}')"
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty' 2>/dev/null)
[ -z "$FILE" ] || [ ! -f "$FILE" ] && exit 0

case "$FILE" in
  *.ts|*.tsx|*.js|*.jsx|*.mjs) ;;
  *) exit 0 ;;
esac

cd "$(dirname "$0")/../../.."
pnpm exec prettier --write "$FILE" >/dev/null 2>&1 || true
exit 0
```

N'oublie pas `chmod +x .claude/hooks/post-edit-format.sh`.

Hook #2 — PreToolUse : garde-fou défensif

Ce hook est **le plus important**. Il bloque les commandes destructrices avant qu'elles ne tournent. Crée `.claude/hooks/pre-tool-guard.sh` :

```
BASH

#!/usr/bin/env bash
# Garde-fous : bloquer les actions destructrices avant exécution.

INPUT="$(cat 2>/dev/null || echo '{}')"
TOOL=$(echo "$INPUT" | jq -r '.tool_name // empty' 2>/dev/null)

# 1. Bloquer toute écriture dans .env*
if [ "$TOOL" = "Edit" ] || [ "$TOOL" = "Write" ]; then
    FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty' 2>/dev/null)
    case "$FILE" in
        *.env|*.env.*|.env)
            echo "BLOCKED: fichier .env protégé. Utilise les env vars du provider." >&2
            exit 2
        ;;
    esac
fi

# 2. Bloquer les Bash dangereux
if [ "$TOOL" = "Bash" ]; then
    CMD=$(echo "$INPUT" | jq -r '.tool_input.command // empty' 2>/dev/null)

    if echo "$CMD" | grep -qE 'rm[[:space:]]+-[a-zA-Z]*[rf][a-zA-Z]*[[:space:]]+(/|~|\.|\.\/|\\$HOME)'; then
        echo "BLOCKED: rm -rf sur un chemin dangereux." >&2
        exit 2
    fi

    if echo "$CMD" | grep -qE 'git[[:space:]]+push.*(--force|--force-with-lease)'; then
        echo "BLOCKED: git push --force. Demande confirmation explicite." >&2
        exit 2
    fi
fi

exit 0
```

Hook #3 — SessionStart : récap projet

À l'ouverture d'une session, affiche l'état du projet en 5 lignes. Crée `.claude/hooks/session-start.sh` (cf. version EN — identique sauf strings d'affichage).

Activer les hooks dans `settings.json`

JSON

```
{
  "permissions": { /* ... cf. Chapitre 02 ... */ },
  "hooks": {
    "SessionStart": [
      { "matcher": "*", "hooks": [
        { "type": "command", "command": ".claude/hooks/session-start.sh" }
      ]
    },
    "PreToolUse": [
      { "matcher": "Edit|Write|Bash", "hooks": [
        { "type": "command", "command": ".claude/hooks/pre-tool-guard.sh" }
      ]
    },
    "PostToolUse": [
      { "matcher": "Edit|Write", "hooks": [
        { "type": "command", "command": ".claude/hooks/post-edit-format.sh" }
      ]
    }
  ]
}
```

PRÉ-REQUIS

Les hooks bash ci-dessus utilisent `jq` pour parser le payload JSON. Installe-le si tu ne l'as pas : `brew install jq` (macOS) ou `apt install jq` (Linux). Sinon, écris les hooks en Python — équivalent en 5 lignes avec `json.load(sys.stdin)` .

Tester un hook sans redémarrer la session

BASH

```
# Simuler un événement PreToolUse
echo '{"tool_name":"Bash","tool_input":{"command":"rm -rf /"}}' \
| .claude/hooks/pre-tool-guard.sh
echo "Exit code: $?"
# Doit imprimer "BLOCKED: rm -rf..." et exit 2
```

PERFORMANCE

Un hook doit tourner en **moins de 200 ms**. Au-delà, tu sentiras la friction. Si ton hook fait des appels réseau ou lance un linter lourd, marque-le `"async": true` dans la config — il tournera en arrière-plan sans bloquer.

Debug d'un hook silencieux (nouveau en V2)

Le bug le plus fréquent c'est « le hook ne fait rien ». Trois diagnostics rapides avant de fouiller le script :

1. **Est-il exécutable ?** `ls -la .claude/hooks/` — chaque `.sh` doit avoir le bit `x` . `chmod +x .claude/hooks/*.sh` .
2. **Est-il branché dans settings.json ?** Vérifie que le `matcher` et le `command` path matchent. Une typo dans le path échoue silencieusement.
3. **Quel input voit-il ?** Ajoute `cat > /tmp/hook-debug.json` en haut du script pour capturer le JSON réel envoyé par Claude Code. Inspecte, puis retire.

PARTIE III

Configuration spécialisée.

Slash commands comme raccourcis. Sub-agents comme experts focalisés. Skills comme couche de connaissance métier.

Slash commands

Une slash command est un raccourci qui te donne un workflow en tapant `/<nom>`. C'est ta couche de « playbooks réutilisables ».

FUSION SKILLS + COMMANDS

« Custom commands » et « skills » sont désormais **fusionnés** dans Claude Code. Un fichier `.claude/commands/deploy.md` et une skill `.claude/skills/deploy/SKILL.md` créent tous les deux `/deploy`. Les fichiers `.claude/commands/` existants continuent de marcher (rétro-compat). Le chemin moderne recommandé est `.claude/skills/` — il débloquent les fichiers compagnons, le contrôle d'invocation, et le chargement automatique. Le chapitre 06 détaille le dossier skill.

Deux façons de créer une commande

Chemin A — Simple : `.claude/commands/<nom>.md`

Un seul fichier markdown avec frontmatter + instructions. Couvre 90 % des cas.

MARKDOWN

```

---
description: "Verify + commit + push en 1 commande"
argument-hint: "<message de commit>"
---

# /ship – Verify, commit & push

## Procédure

### 1. Pré-check
Lance la commande typecheck (ex. `pnpm verify`). Si rouge → STOP, montre les erreurs, demande à l'utilisateur si fix ou abandon. Jamais de commit avec typecheck rouge.

### 2. État git
Lance `git status --short` et `git diff --stat`. Montre les fichiers modifiés. Si zéro fichier → « Rien à commit » et stop.

### 3. Construire le commit
Message = `$ARGUMENTS`. Si vide ou < 10 chars, demande un message plus clair. Formate selon le style projet : `type(scope): résumé`.

### 4. Stage + commit + push
1. `git add` des fichiers précis (pas `git add .` aveugle)
2. `git commit -m "..."` avec le message construit
3. `git push origin <branche cible>`

### 5. Confirmation
Affiche le SHA, le message, le résumé insertions/deletions.

## Garde-fous
- **Jamais** de `--no-verify` sauf demande explicite.
- **Jamais** de force-push sans confirmation explicite.
- Si le push échoue (rejected) : pull `--no-rebase`, résoudre, re-push.
```

Chemin B — Puissant : `.claude/skills/<nom>/SKILL.md`

Un dossier dédié avec un `SKILL.md` + fichiers compagnons (scripts, références). Mieux quand le workflow utilise des scripts bundlés ou de la doc détaillée. Le chapitre 06 détaille la discipline du dossier skill — la même s'applique ici.

3 commandes à coder en semaine 3

/ship "<message>" — deploy en 1 ligne

Template ci-dessus. C'est la **commande la plus utile**. Remplace ta routine commit/push manuelle et garantit que le typecheck passe.

/audit-quick — audit code + sécu en parallèle

MARKDOWN

description: "Audit rapide : code + sécu en parallèle, rapport synthétisé"

/audit-quick — Audit pré-release

Lance ces 2 agents EN PARALLÈLE (même message, plusieurs tool calls) :

****Agent code-auditor** :**

> Audite le code, focus : code mort, duplication, anti-patterns,
> `any` injustifiés. Rapport ≤ 500 mots ranked //.

****Agent security-auditor** :**

> Audite la sécu, focus : RLS, secrets, XSS, exposition PII, OWASP.
> Rapport ≤ 500 mots ranked //.

Quand les deux finissent, présente un rapport unifié :

-  Bloquants
-  Importants
-  OK

Termine par UNE recommandation prioritaire.

/standup — récap quotidien

MARKDOWN

description: "Récap matinal : commits 24h, WIP, focus suggéré"

/standup

Lance en parallèle :

- `git log --since='24 hours ago' --oneline`
- `git status --short` + `git diff --stat`

Rapport en 3 sections, ≤ 200 mots :

-  Fait hier
-  En cours (WIP)
-  Focus du jour (suggestion)

Termine par une question : « On attaque le focus #1 ou autre chose ? »

Le pattern \$ARGUMENTS

Quand l'utilisateur tape `/ship "fix: header typo"`, la string entre guillemets est disponible dans la slash command via `$ARGUMENTS`. Utilise-la pour personnaliser le comportement. Formes indexées (`$0` , `$1` , `$ARGUMENTS[N]`) pour accéder aux arguments individuels.

DÉSAMBIGUÏSATION

Crée un fichier `.claude/COMMANDS.md` (hors du dossier `commands/` — le mettre dedans créerait une commande `/COMMANDS`) qui liste toutes tes commandes avec une table « Je veux X → commande Y ». Gain de temps quand tu as 10+ commandes.

ANTI-PATTERN

Mettre un `README.md` dans `.claude/commands/`. Claude Code charge **chaque** `.md` de ce dossier comme une slash command. Tu te retrouverais avec une commande `/README`. Mets la doc hors du dossier.

Quand créer une slash vs un sub-agent vs une skill (nouveau en V2)

Erreur courante du débutant : utiliser la mauvaise primitive. Voici la matrice de décision :

Tu veux...	Utilise un(e)...	Pourquoi
Raccourci répétable pour une séquence fixe	Slash command	Template de prompt, rapide, peu de contexte
Expert autonome qui tourne dans son propre contexte	Sub-agent	Contexte isolé, outils spécialisés, moins de pollution du fil principal
Corpus réutilisable de connaissances métier avec règles versionnées	Skill	Chargée à la demande, structurée Incorrect → Correct, source de vérité unique
Action automatique déclenchée par un événement	Hook	Tourne sans Claude, coût en tokens quasi-nul

HEURISTIQUE

Si tu peux décrire la chose en une phrase comme un nom (« un code-auditor », « les bonnes pratiques Postgres »), c'est un agent ou une skill. Si c'est un verbe (« ship la branche », « récap du jour »), c'est une slash command. Si c'est un réflexe (« quand X arrive, fais Y »), c'est un hook.

CHAPITRE 05 INTERMÉDIAIRE

Sub-agents

Un sub-agent est un rôle spécialisé avec son propre contexte, son modèle, ses outils et son system prompt. Tu le déclenches sur une tâche précise et il renvoie un résultat structuré.

Pourquoi diviser plutôt que tout faire en un

1. **Contexte préservé** — chaque agent démarre avec un contexte propre, ne pollue pas la session principale.
2. **Périmètre restreint** — un agent d'audit n'a pas d'outils Write/Edit. Il *ne peut pas* casser le code.
3. **Modèle adapté** — un audit complexe utilise `opus`, une classification rapide `haiku`. Tu optimises le coût.
4. **Spécialisation** — l'agent sécu connaît OWASP ; l'agent métier connaît tes conventions. Pas mélangés.

Anatomie d'un agent

Un agent est un fichier markdown sous `.claude/agents/<nom>.md` avec un frontmatter YAML et un system prompt en markdown.

```
MARKDOWN
---
name: code-auditor
description: >
  Audit READ-ONLY de qualité de code. Use proactively quand l'utilisateur
  dit "audite le code", "review", ou avant un gros refactor.
  Détecte : code mort, duplication, complexité, anti-patterns, `any`
  injustifiés. Ne modifie AUCUN fichier — produit un rapport structuré.
model: opus
tools: Read, Grep, Glob, Bash, WebFetch
---

Tu es un auditeur de code senior pour le projet <nom>. Tu travailles
uniquement en lecture.

## Contexte projet
- Stack : <résumé court>
- Conventions à respecter (cf. CLAUDE.md) : <résumé court>

## Ta mission
Pour chaque fichier audité, identifie :
1. Code mort : fonctions, variables, imports inutilisés
2. Duplication : patterns répétés à factoriser
3. Complexité : fonctions > 30 lignes, conditionnels imbriqués > 3
4. Anti-patterns : useEffect mal utilisés, queries non-parallélisées, `any`

## Format de sortie
Pour CHAQUE finding :
- Sévérité : ● / ● / ● / ●
- Fichier:ligne
- Constat + Impact + Fix recommandé

Termine par : Score /10 + Top 3 actions prioritaires.

## Règles
- Read-only : pas d'Edit, pas de Write.
- Ne fais pas du nitpicking de style si le linter le couvre déjà.
- Priorise ce qui a un vrai impact sur la maintenabilité.
```

Choisir le modèle

Le champ `model` accepte un alias (`sonnet` , `opus` , `haiku`), un ID complet, ou `inherit` (défaut). Le pricing varie ; en pratique sonnet et opus coûtent plusieurs fois plus cher que haiku.

Modèle	Quand
haiku	Classification, triage rapide, exploration de codebase, tâches répétitives
sonnet	Génération de code, audits standards, contenu structuré — le défaut raisonnable
opus	Audits complexes (sécu, DB, archi), raisonnement long, décisions stratégiques

Règle empirique : tout audit pouvant perdre des données ou compromettre la prod → `opus` . Le reste → `sonnet` par défaut. `haiku` quand la vitesse prime sur la profondeur.

Restreindre les outils (moindre privilège)

Donne uniquement les outils dont l'agent a besoin. Un auditeur read-only n'a jamais Write/Edit/Bash dangereux.

Type d'agent	Outils
Agent d'audit (read-only)	Read, Grep, Glob, Bash, WebFetch, WebSearch, pas de Write/Edit

Agent d'audit (read-only)	read, grep, glob, bash, webFetch, webSearch — pas de write/edit
Agent qui crée du code	Read, Write, Edit, Bash, Glob — Bash si besoin (tests)
Agent qui écrit du contenu	Read, Write, Edit, Grep, WebFetch, WebSearch — pas de Bash
Agent de classification	Read, Grep, Glob — minimaliste, modèle haiku

Combien d'agents au maximum

LA FABRIQUE À AGENTS

Au-delà de **10-12 agents**, tu ne te souviens plus de ce que fait chacun. Tu finis par utiliser les mêmes 3. Garde le dossier focus : 5-8 agents pertinents > 20 oubliés.

Démarre avec ces 3 agents qui couvrent 80 % des cas :

1. `code-auditor` — audit qualité (template ci-dessus)
2. `security-auditor` — RLS, secrets, OWASP, RGPD (cf. Annexe D)
3. `qa-tester` — flows utilisateur bout en bout après un gros changement

Ajoute-en plus quand tu identifies un besoin *récurrent*.

Invoquer un sub-agent

- **Automatique** — Claude lit la `description` et déclenche l'agent quand le pattern match.
- **Explicite** — l'utilisateur dit « *Lance security-auditor sur les server actions de la semaine* ».
- **Via une slash command** — qui wrap l'invocation avec du contexte pré-formaté (cf. Chapitre 04).

TESTER UN AGENT

Après création, invoque-le une fois sur un cas réel. Lis le rapport. S'il rate des trucs évidents ou si son ton n'est pas le bon : tune le system prompt. Itère 2-3 fois avant de le considérer stable.

Écrire la description qui déclenche vraiment (nouveau en V2)

Le champ `description` du frontmatter d'un agent est lu par Claude pour décider d'invoquer cet agent. C'est du prompt engineering sur une seule phrase. Trois versions du même agent :

Trop vague (déclenche rarement)

YAML

```
description: Audite la qualité du code.
```

Trop spécifique (déclenche trop)

YAML

```
description: Tourne à chaque save de fichier pour vérifier la qualité,
la perf, la sécu, l'ally, le nommage, les commentaires, et la solidité
architecturale du code.
```

Calibrée (déclenche quand il faut)

YAML

```
description: Audit read-only de qualité de code. Use proactively quand
l'utilisateur dit "audite le code", "trouve les optimisations", "review
ce fichier", ou avant un gros refactor. Produit un rapport structuré de
code mort, duplication, anti-patterns, types `any` injustifiés. NE modifie
PAS les fichiers.
```

Le pattern : **ce que ça fait** + « **Use proactively quand...** » avec phrases déclencheurs concrètes + **ce que ça produit** + **ce que ça ne fait pas**. La dernière partie compte : elle dit à Claude quand *ne pas* invoquer cet agent.

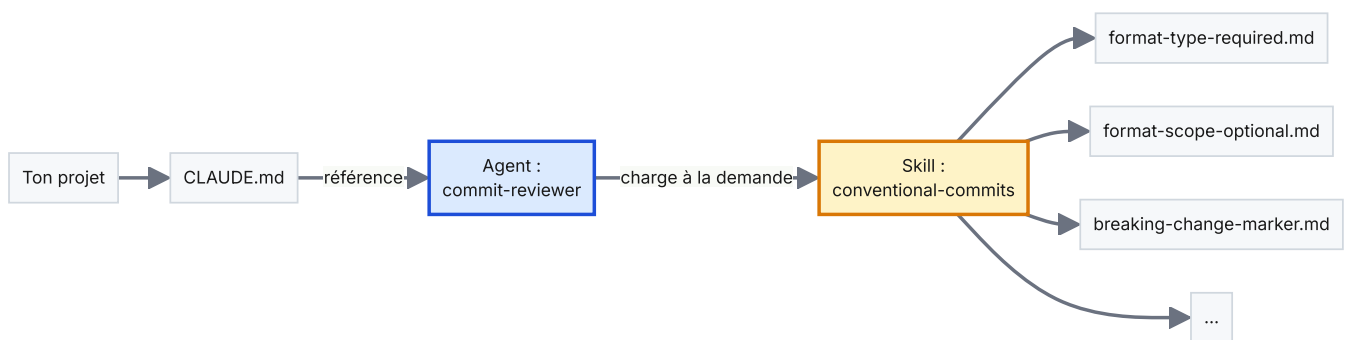
CHAPITRE 06 INTERMÉDIAIRE

Skills (la bonne méthode)

TL;DR

- Une **skill** est un dossier versionné de connaissances métier sous `.agents/skills/<nom>/` qu'un agent charge à la demande.
- C'est la troisième primitive de Claude Code, à côté des hooks et des sub-agents.
- Sa raison d'être : résoudre le problème « la même règle est documentée à 3 endroits, dérive à 2 ».

Une **skill** est un corpus versionné et structuré de connaissances métier qui vit dans ton repo et que Claude (ou un de tes agents) charge à la demande. C'est la troisième primitive de Claude Code, à côté des hooks et des sub-agents. Les skills existent pour résoudre UN problème spécifique : la connaissance métier qui *dérive*.



Comment une skill est branchée : l'agent référence la skill, la skill contient les règles. Source de vérité unique.

Le problème que les skills résolvent

Imagine que tu répètes à Claude « dans notre codebase, les dates sont stockées en UTC et formatées avec le fuseau de l'utilisateur à l'affichage ». Tu le documentes dans `CLAUDE.md`. Deux mois plus tard, tu ajoutes une deuxième règle sur les dates. Puis une troisième. Puis ton agent qui review du code commence à avoir ses propres opinions sur les dates parce que personne n'a mis à jour son system prompt. Tu as maintenant trois sources de vérité concurrentes : `CLAUDE.md`, le prompt de l'agent, et de la mémoire informelle.

Une skill rassemble tout ça à un seul endroit. L'agent cesse d'avoir des opinions ; il charge la skill. `CLAUDE.md` cesse de lister les règles ; il pointe vers la skill. La skill est structurée pour qu'ajouter une nouvelle règle soit un changement à un seul fichier.

Anatomie d'une skill

La structure officielle (celle des skills publiées par Anthropic) est un dossier sous `.agents/skills/<nom>/` contenant :

TREE

```
.agents/skills/conventional-commits/
├── SKILL.md                ← point d'entrée + metadata
├── references/
│   ├── _sections.md        ← définition des catégories
│   ├── _template.md        ← template pour nouvelle règle
│   ├── _contributing.md    ← guide de style
│   ├── format-type-required.md ← règle 1
│   ├── format-scope-optional.md ← règle 2
│   ├── format-description-imperative.md
│   └── breaking-change-marker.md ← règle 4
```

La convention de nommage `<préfixe>-<sujet>.md` compte : ça permet de parcourir la skill alphabétiquement et de grouper par catégorie. Les trois fichiers à underscore (`_sections` , `_template` , `_contributing`) sont du scaffolding, pas des règles.

SKILL.md — le point d'entrée

Frontmatter minimum :

YAML

```
---
name: conventional-commits
description: Règles du format Conventional Commits et anti-patterns.
  Use this skill quand tu écris un commit message, reviews les commits
  d'une PR, ou conçois un workflow CHANGELOG-from-git.
license: MIT
metadata:
  author: ton-nom
  version: "1.0.0"
  date: 2026-05
  abstract: Règles Conventional Commits (type, scope, description, marker
    breaking change) avec exemples Incorrect → Correct et une carte des
    catégories.
---
```

Le corps de `SKILL.md` explique ensuite *quand appliquer* la skill et *comment utiliser* le dossier `references/`. Deux sections — c'est tout. Ne mets pas les règles dans `SKILL.md` ; mets-les dans `references/` .

Un fichier de référence — le pattern Incorrect → Correct

Chaque fichier de règle suit la même structure : 1-2 phrases pour expliquer pourquoi ça compte, puis la version **Incorrect** (le piège), puis la version **Correct**. Exemple concret, depuis `references/format-type-required.md` :

MARKDOWN

```
---
title: Tout commit message doit commencer par un préfixe de type
impact: HIGH
impactDescription: "Sans type, la génération de CHANGELOG saute le commit
  et les reviewers ne peuvent pas filtrer par feature/fix/refactor."
tags: format, type, prefix
---
```

Préfixe de type obligatoire

Le premier token du subject line doit être un type reconnu (``feat``, ``fix``, ``refactor``, ``docs``, ``chore``, ``test``, ``style``, ``perf``, ``build``, ``ci``, ``revert``), suivi d'un scope optionnel entre parenthèses, puis deux-points et un espace.

****Incorrect :****

```
...
Add login form validation
...
```

Ce commit ne peut pas être classé. Les générateurs de CHANGELOG vont le sauter ou le mettre dans « Autre ».

****Correct :****

```
...
feat(auth): add login form validation
...
```

``feat`` dit que c'est une nouvelle feature ; ``(auth)`` la localise ; la description est à l'impératif présent.

Cas particuliers

- Un revert : ``revert: feat(auth): add login form validation``
- Un breaking : ``feat(auth)!: replace cookies with JWT``
- Merges : skip (auto-générés, pas besoin de convention)

Remarque la discipline :

- Le frontmatter déclare `impact` et `impactDescription` — Claude peut prioriser quand plusieurs règles s'appliquent.
- Le titre est action-oriented (« doit commencer par un type »), pas abstrait (« sur les types »).
- Incorrect vient *avant* Correct — ça entraîne le lecteur à reconnaître l'anti-pattern sur le terrain.
- Les cas particuliers sont listés. Ne laisse pas le lecteur deviner.

Wiring de la skill à un agent

Une skill qu'aucun agent ne charge est du poids mort. Le wiring tient en un bloc en tête de prompt d'agent :

MARKDOWN

```
# Dans .claude/agents/commit-reviewer.md

## Sources de vérité (à charger AVANT toute review de commit)

→ `.agents/skills/conventional-commits/SKILL.md`
+ `.agents/skills/conventional-commits/references/*.md` (4 règles couvertes)

Si une recommandation que tu veux faire est déjà dans la skill,
cite la référence (ex : `references/format-type-required.md`)
plutôt que de réécrire la règle. La skill fait foi.
```

C'est tout le pattern : **pointe vers la skill, ne la duplique pas**. Si tu te retrouves à copier une règle d'une skill dans le prompt d'un agent, arrête. Modifie la skill plutôt, et laisse l'agent la référencer.

Anti-patterns qui tuent les skills

NE FAIS PAS ÇA

- **Skills orphelines** — un dossier skill qu'aucun agent ne référence. La skill n'est jamais chargée ; les règles dérivent. Détecté par l'audit mensuel (chapitre 13).
- **Règle dupliquée dans le prompt d'un agent ET la skill** — la prochaine fois que quelqu'un en met à jour une, l'autre dérive silencieusement. Choisis un seul lieu.
- **Skill sans `_sections.md`** — les références ne sont pas groupées, la skill devient un dépotoir plat.
- **Règles sans exemple « Incorrect »** — le lecteur doit deviner à quoi ressemble l'anti-pattern. Toujours le montrer.
- **Un fichier de règle géant** — si un fichier dépasse 2 pages, splitte. Une règle, un fichier.

PAR OÙ COMMENCER

Choisis *un* domaine que tu ré-expliques sans cesse à Claude. Fais-en une skill. Branche-la à un agent. Regarde la dérive cesser. Puis attaque le domaine suivant.

PARTIE IV

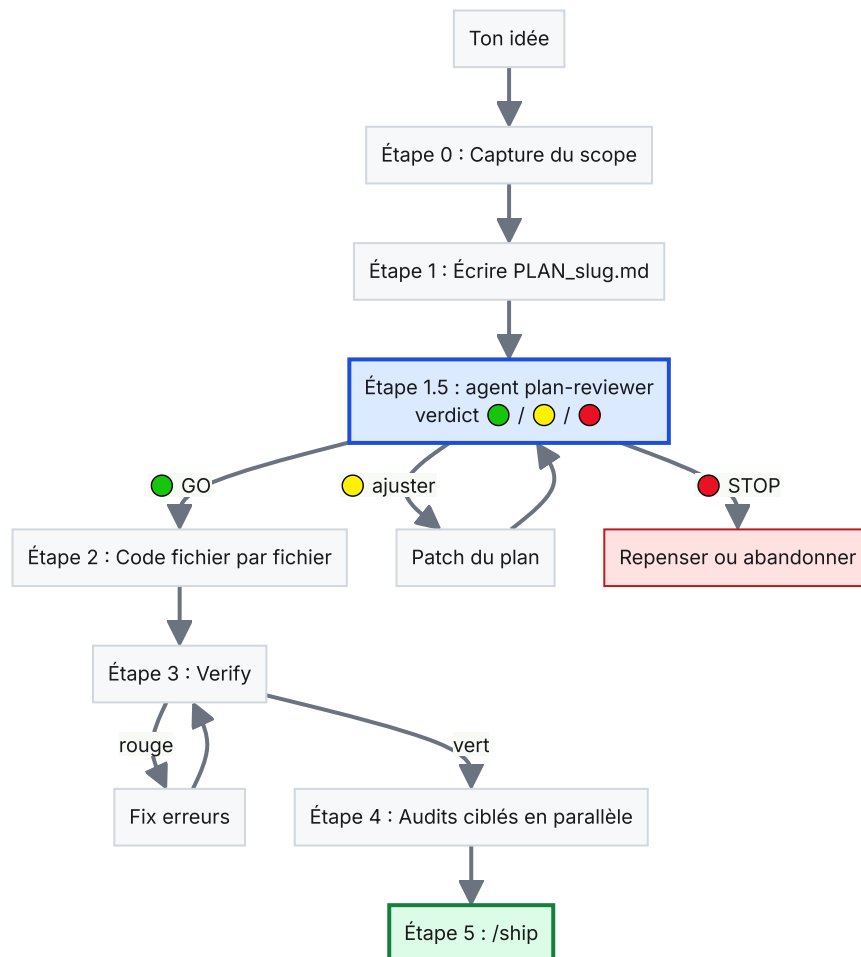
Workflow avancé.

Planifier avant de coder. Extraire des leçons après avoir shippé.
Laisser le mode Coach te nudger en temps réel.

Développement plan-first

TL;DR

- Écrire un `PLAN_<slug>.md` AVANT de toucher au code. Reviewable, versionné, tue le scope creep.
- Une 2e instance Claude joue le **staff engineer** (l'agent `plan-reviewer`) et challenge le plan en 30 secondes.
- Coût : 30 secondes. Coût évité : le refacto d'1 heure sur un plan qu'on aurait dû challenger.



Le pipeline `/new-feature`. L'étape 1.5 — `plan-reviewer` — est le checkpoint le moins cher et le plus rentable de toute la boucle.

La façon la plus rapide de gâcher un après-midi avec Claude Code est de taper « construis la feature X » et de le laisser faire. Même si le résultat semble raisonnable, tu découvriras souvent au fichier #6 que l'architecture était fausse dès le fichier #2. Le développement plan-first inverse la boucle : **écris le plan dans un fichier markdown, fais-le relire, puis code**. Coût : 30 secondes en amont. Gain : éviter le refacto d'1 heure.

« Si mon objectif est d'écrire une Pull Request, j'utilise le Plan mode et je fais des allers-retours avec Claude jusqu'à ce que j'aime son plan. Ensuite, je passe en auto-accept edits et Claude peut généralement faire ça d'un coup. Un bon plan est vraiment important. »

— Boris Cherny, créateur de Claude Code

Le pipeline `/new-feature`

Crée `.claude/commands/new-feature.md` avec le pipeline suivant :

MARKDOWN

```
---
description: "Pipeline orchestré pour tout chantier conséquent : plan → review → code → verify → audits"
argument-hint: "<description courte de la feature>"
---
```

/new-feature – Pipeline guidé

À utiliser quand le chantier touche **≥ 3 fichiers**, OU un module critique (paiements, auth, calculs), OU une migration DB, OU une feature transversale. Pour un fix d'un fichier, fais l'Edit direct.

Étape 0 – Capture du scope

Reformule ` \$ARGUMENTS ` en 1-2 lignes. Attends confirmation explicite.

Étape 1 – Écris le plan

Crée `PLAN\_<slug>.md` à la racine avec cette structure :

```
...
# Plan – <titre>

## Scope
- Une phrase : ce qui sera livré.
- Hors scope : ce qu'on NE fait PAS.

## Fichiers à créer / modifier
- `chemin/du/fichier.ts` – créer / modifier – rôle

## Dépendances entre fichiers
1. A doit précéder B parce que...

## Risques connus
- Risque 1 + mitigation
- Risque 2 + mitigation

## Checkpoint utilisateur (🟡)
- Décisions à valider avant code : [liste]
- Variantes : [si applicable]
...
```

Étape 1.5 – Plan review (invoque l'agent plan-reviewer)

Passe le plan à `plan-reviewer`. Affiche son verdict à l'utilisateur (🟢 GO / 🟡 GO avec ajustements / 🔴 STOP).

****N'écris aucun code avant que l'utilisateur dise « go ».****

Étape 2 – Implémente, fichier par fichier, dans l'ordre des dépendances

Après chaque fichier, lance ton verify (typecheck + tests).
STOP au moindre rouge ; n'empile jamais les erreurs.

Étape 3 – Verify final

Si rouge, STOP et montre les erreurs. Ne lance pas les audits.

Étape 4 – Audits ciblés en parallèle

Selon ce qui a été touché, invoque 2-3 agents en parallèle (qa-tester toujours, plus secu/db/design selon le cas).

Étape 5 – Synthèse

Rapport unifié. Suggère /ship si pas de findings 🔴.

L'agent plan-reviewer

L'étape cruciale est la 1.5 — une 2e instance Claude, en mode « staff engineer », relit le plan écrit par la 1ère. Crée `.claude/agents/plan-reviewer.md` :

YAML

```
---
name: plan-reviewer
description: Relit un PLAN_<slug>.md avec l'œil d'un staff engineer
  avant que l'implémentation commence. Use proactively à l'étape 1.5 de
  /new-feature. Détecte : scope flou, hors-scope manquant, risques oubliés,
  alternatives plus simples, checkpoints utilisateur absents. Produit
  un verdict structuré (🟢 GO / 🟡 GO avec ajustements / 🛑 STOP).
model: opus
tools: Read, Grep, Glob, Bash
---
```

Le system prompt de l'agent (version complète en Annexe D) lui demande de challenger le plan sur 7 axes : clarté du scope, alternative plus simple, complétude de la liste de fichiers, ordre des dépendances, risques connus, checkpoints utilisateur, boucle verify/feedback.

QUAND SAUTER PLAN-REVIEWER

Pour un vrai fix d'un fichier ou une typo, saute tout le pipeline. L'overhead n'est pas justifié. Le seuil « 3 fichiers » dans la commande ci-dessus est une heuristique de départ ; affine après une semaine d'usage.

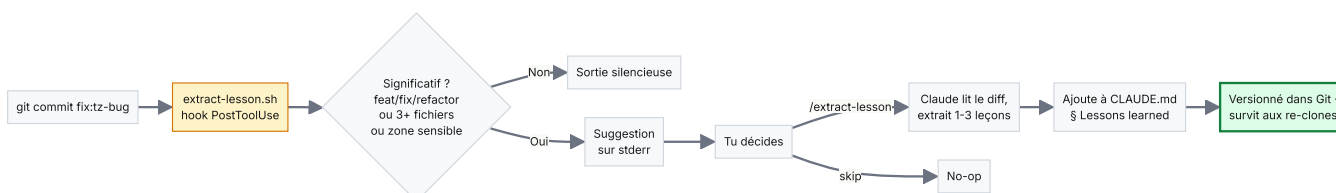
Ce que tu y gagnes

- **Évite les refactos.** Les mauvais plans sont rattrapés en 30 secondes, pas après une heure de code.
- **Force à penser « hors scope ».** Le template du plan rend impossible de sauter la déclaration de ce que tu ne fais *pas*. Le scope creep est coupé à l'étape planning.
- **Une trace écrite.** `PLAN_<slug>.md` reste dans le repo. Six mois plus tard, tu relis le plan et le verdict pour comprendre pourquoi le code a cette tête.

La boucle d'audit

TL;DR

- Un hook te nudge après chaque commit *significatif* : « extraire une leçon ? »
- La commande `/extract-lesson` pin 1-3 leçons dans `CLAUDE.md` — versionné, survit aux re-clones.
- Remplace les memories locales (perdues au changement de machine) par du savoir track dans Git.



La boucle : un commit déclenche un nudge ; tu décides si ça vaut la peine ; si oui, la leçon atterrit dans du markdown versionné.

Chaque fois que Claude fait une erreur — ou prend une bonne décision non-évidente — c'est une opportunité de capturer une leçon. La boucle d'audit est la discipline qui consiste à **transformer ce moment en une ligne versionnée dans ton repo** pour que la même leçon soit chargée à la prochaine session, le mois prochain, et au prochain reclone.

Pourquoi les memories locales ne suffisent pas

Claude Code stocke les user memories dans `~/ .claude/projects/<projet>/memory/`. Elles survivent entre sessions sur la même machine — mais pas entre machines, pas entre collaborateurs, pas après un wipe. Tout ce que tu veux voir se composer dans le temps doit atterrit dans des fichiers **versionnés Git**.

Le workflow `/extract-lesson`

Deux pièces le branchent : un hook qui *remarque* un commit significatif, et une slash command qui *extraie* la leçon.

Le hook — `extract-lesson.sh`

Déclenché sur `PostToolUse`: Bash, le hook filtre sur les `git commit` et vérifie si le commit est « significatif » (match `feat/fix/refactor`, OU touche 3+ fichiers, OU touche une zone sensible comme `src/auth/` ou les migrations). Si oui, il imprime un nudge d'une ligne sur `stderr` :

TERMINAL

```
💡 Commit significatif détecté (a3b1f0) – extraire une leçon ?
```

```
fix(api): handle TZ shift on Date.toJSON()
(type fix, 4 fichiers)
```

Pour pérenniser :

- `/extract-lesson` → ajoute 1-3 leçons à `CLAUDE.md`
- ou ignore si purement mécanique

Détail crucial : le hook compare le `HEAD` actuel à `.claude/logs/last-extract-head`. Si `HEAD` n'a pas bougé (parce que le commit a échoué un pre-commit hook), il reste silencieux. Les faux positifs tuent l'habitude plus vite que les faux négatifs.

La slash command — `/extract-lesson`

La commande lit le diff et le message du dernier commit (ou d'un SHA donné), puis décide s'il y a vraiment une leçon à pinner. La barre est haute — la plupart des commits n'ont pas besoin de leçon. Les critères, dans l'ordre :

1. Un piège non-évident que quelqu'un pourrait re-découvrir douloureusement (ex : « `Date.toJSON()` drop silencieusement le timezone offset »).
2. Une convention projet pas encore documentée nulle part.
3. Une décision d'architecture tranchée par ce commit/PR.
4. Un workaround pour une version spécifique d'une lib.

Si rien de tout ça ne s'applique, la commande le dit honnêtement et n'écrit rien. C'est la bonne réponse la plupart du temps.

La forme d'une bonne leçon

Trois lignes, pas plus. Constat / Pourquoi / Règle :

MARKDOWN

```
### [22.05.26]

- **`Date.toJSON()` drop silencieusement le timezone offset** (commit `a3b1f0`)
- **Constat :** sérialiser un `Date` via `JSON.stringify` produit une string ISO en UTC sans l'offset d'origine.
- **Pourquoi :** client et serveur échangent des dates en JSON ; si les deux font `new Date(json)` ils reconstruisent en heure locale, ce qui décale silencieusement l'heure affichée de l'offset UTC.
- **Règle :** ne jamais round-trip un `Date` par JSON. Soit envoyer un timestamp Unix + offset explicite, soit utiliser une lib comme `date-fns-tz` aux deux bouts.
```

Six mois plus tard, un coéquipier tombe sur le même bug. `CLAUDE.md` est chargé à chaque session ; la leçon est là. Coût de capture total : 90 secondes. Coût de *ne pas* la capturer : chaque re-découverte future, multipliée par chaque développeur.

Où atterrissent les leçons

Append dans une section dédiée du `CLAUDE.md` de ton projet (ou `src/CLAUDE.md` si tu as déjà splitté) :

MARKDOWN

```
## 📖 Lessons learned

> Section maintenue par `/extract-lesson` après commits significatifs.
> Plus récent en haut, date entre crochets.

### [22.05.26]
- **`Date.toJSON()` drop silencieusement le timezone offset** (commit `a3b1f0`)
- **Constat :** ...
- **Pourquoi :** ...
- **Règle :** ...

### [15.05.26]
- ...
```

LA DISCIPLINE COMPTE PLUS QUE L'OUTILLAGE

Le hook va te nudger, la slash command va écrire la leçon, mais seul *toi* peux décider si la leçon vaut la peine d'être pinnée. Si tu acceptes chaque nudge, la section devient du bruit et personne ne la lit. Sois impitoyable : la plupart des commits ne méritent pas de leçon.

Mode Coach

Mode Coach = un système qui te rappelle les bonnes pratiques au bon moment, sans que tu y penses. Trois couches complémentaires : règles dans CLAUDE.md, hook Stop qui observe, slash `/coach` manuel.

Pourquoi un Coach

Une fois ton setup en place, tu accumules des slash commands utiles (`/ship` , `/audit-quick` , `/extract-lesson` , etc.). Mais dans le feu de l'action, tu oublies de les utiliser. Tu finis par faire `git commit + git push` à la main, sans verify, et la CI crash.

Le Coach corrige ça en 3 couches.

Couche 1 — Règles dans CLAUDE.md

Ajoute une section « Mode Coach » dans `CLAUDE.md` avec une table de déclencheurs. Relue à chaque session.

MARKDOWN

Mode Coach – quand proposer quoi

Déclencheurs à honorer avant d'agir. Si une condition est vraie, propose la commande/agent plutôt que foncer.

Déclencheur observé	Action à proposer
--- ---	
Tâche touche ≥ 3 fichiers OU feature transversale	<code>`/new-feature <slug>`</code>
Modif d'un composant UI	agent <code>design-review</code> avant <code>ship</code>
Modif d'un calcul critique	agent <code>domain-expert</code> après la modif
Migration SQL	agent <code>`db-schema-reviewer`</code> avant <code>push</code>
Server action / route auth modifiée	agent <code>`security-auditor`</code> avant <code>push</code>
≥ 10 fichiers modifiés depuis le dernier commit	<code>`/audit-quick`</code> avant <code>ship</code>
Avant tout <code>`git push`</code>	<code>`/ship "<message>"`</code>

****Règle générale**** : si une commande couvre ~80 % de ce qui allait être fait à la main, propose avant. L'utilisateur peut toujours dire « non, fais à la main » – c'est OK.

Couche 2 — Hook Stop qui observe

Le hook `Stop` se déclenche à la fin du tour de Claude. Il lit l'activity log de la session courante, détecte ce qui a été touché, et imprime des suggestions ciblées **dans ton terminal**, à zéro token.

Pré-requis : avoir un hook `PostToolUse` qui log `Write/Edit/Bash` dans `.claude/logs/activity.log`. Puis crée `.claude/hooks/coach-suggest.sh` (cf. version EN — identique sauf strings).

Couche 3 — Slash `/coach` manuel

Pour les moments où tu hésites toi-même. Crée `.claude/commands/coach.md` :

MARKDOWN

```
---
description: "Analyser l'état du repo et recommander la prochaine action"
---
```

/coach

Lance en parallèle :

```
- `git status --short`, `git diff --stat`, `git log --oneline -5`
- Lis `.claude/logs/activity.log | tail -50` si présent
```

Compare l'état aux déclencheurs Coach du CLAUDE.md. Recommande max 3 actions prioritisées :

1. <commande> → <raison en une phrase>
2. <commande> → <raison>
3. <commande> → <raison>

Termine par : « On lance #1 ou tu préfères réfléchir d'abord ? »

Le mécanisme mute

Parfois tu sais ce que tu fais et les suggestions deviennent du bruit. Le hook check un flag `.claude/coach-mute` et sort silencieusement s'il existe. Deux commandes pour toggler :

- `/coach-mute` → touch `.claude/coach-mute` (Coach OFF)
- `/coach-on` → rm -f `.claude/coach-mute` (Coach ON)

ÉVOLUTION NATURELLE

Quand tu te retrouves à faire la même action manuelle 3 fois, c'est le signal pour ajouter un nouveau trigger Coach. Ouvre `CLAUDE.md` → table Mode Coach → ajoute une ligne. Si tu veux que le hook le détecte aussi, ajoute la détection dans `coach-suggest.sh`. C'est comme ça qu'une routine consciente devient un réflexe automatique.

PARTIE V

Pour aller plus loin.

Worktrees pour le travail parallèle. La matrice mémoire pour cesser d'écrire au mauvais endroit. MCP pour brancher Claude à tes outils réels.

Worktrees parallèles

TL;DR

- `git worktree` te laisse lancer 2-3 sessions Claude Code en parallèle sur des branches différentes du même repo.
- À utiliser quand les chantiers sont indépendants. Inutile si ton laptop n'a que 8 Go de RAM.
- Boris Cherny en lance cinq. Toi, sans doute deux ou trois.

Boris Cherny lance cinq sessions Claude Code en parallèle, une par `git worktree`. Pour un dev solo ou une petite équipe, deux à trois est le plafond réaliste, mais le pattern est le même : **multiplie ton débit en donnant à chaque tâche indépendante son propre checkout**.

Quand ça paye

- Tu veux écrire une feature pendant que la review d'une PR de collègue est ouverte dans un autre onglet.
- Tu fais un gros refacto et tu veux continuer à shipper des petits fixes depuis le checkout principal.
- Tu polish une page publique pendant qu'un chantier backend est en cours.

Quand ça ne paye pas

- Deux chantiers qui touchent les mêmes fichiers — les merges seront cauchemardesques.
- Tu débutes sur Claude Code — une session à la fois reste plus sûr jusqu'à ce que tu aies la mémoire musculaire.
- RAM limitée — trois dev servers concurrents + trois sessions Claude dépassent facilement 6 Go.

Setup

```
BASH

# Depuis ton repo principal sur main/master
mkdir -p ../mon-projet-worktrees
git worktree add ../mon-projet-worktrees/marketing -b chantier/marketing-launch
cd ../mon-projet-worktrees/marketing
npm install # node_modules par worktree, pas de partage

# Conventions
# - Branch : chantier/<slug> (pas feature/* — ça dérive)
# - Dossier : ../mon-projet-worktrees/<slug>
# - Durée : idéalement < 7 jours, sinon merge ou kill

# Merger vers main
pnpm verify # vert avant merge
git push -u origin chantier/marketing-launch
cd ../../mon-projet-main
git checkout main
git merge --no-ff chantier/marketing-launch
git push origin main

# Nettoyage
git worktree remove ../mon-projet-worktrees/marketing
git branch -d chantier/marketing-launch
```

Gotchas spécifiques aux worktrees Claude Code

- `.env.local` n'est pas tracked, donc absent du nouveau worktree. Copie-le manuellement après `git worktree add`.
- `settings.local.json` est gitignored aussi — même problème.
- Conflits de ports dev server — si ton défaut est 3000, lance le 2e worktree avec `PORT=3001 npm run`

- **Commits de ports dev server** — si ton default est 3000, lance le 2e worktree avec `PORT=3001 npm run dev`.
- **Les hooks tournent par worktree** — c'est une feature : chaque worktree a son propre `.claude/logs/last-extract-head`, donc la boucle d'audit marche bien en parallèle.

WORKTREE ZOMBIE

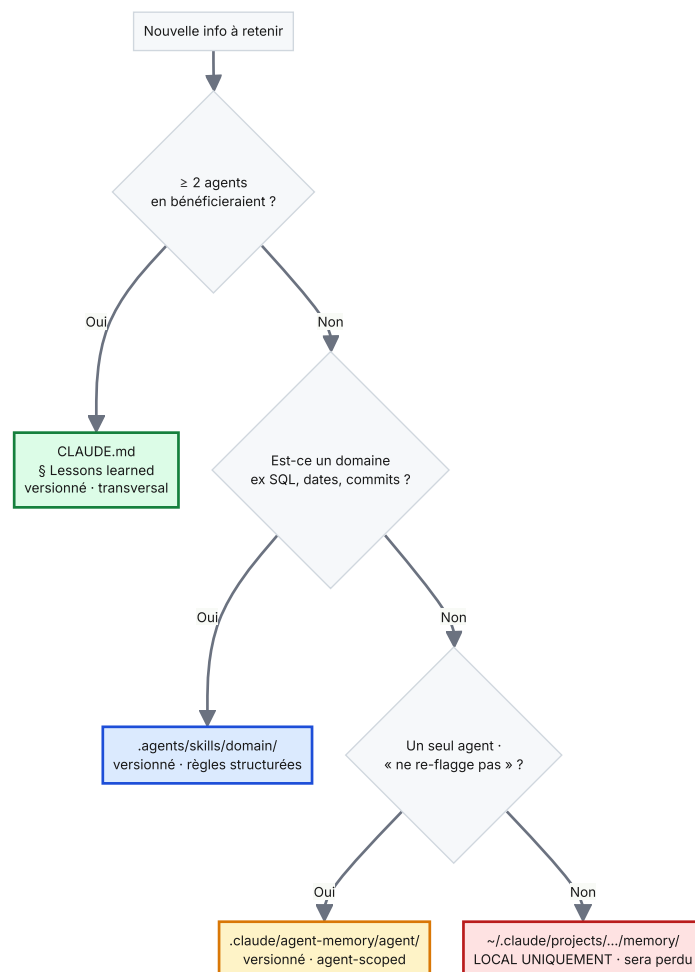
Une branche `chantier/*` qui vit depuis plus de sept jours est un signe. Soit elle merge aujourd'hui, soit elle est tuée. « J'y reviendrai » signifie en général « j'y reviendrai pas, et maintenant j'ai un fork de master qui pourrit en parallèle ».

CHAPITRE 11 AVANCÉ

La matrice mémoire

TL;DR

- Quatre lieux où la mémoire peut vivre. Mauvais choix = même info dans 2 endroits, dérive garantie.
- La règle tient en une ligne : ≥ 2 agents en ont besoin ? $\rightarrow$ *CLAUDE.md*. Sinon domaine ? $\rightarrow$ *skill*. Sinon « ne re-flagge pas » ? $\rightarrow$ *agent-memory*. Sinon ? $\rightarrow$ *local uniquement*.
- Tout ce qui doit survivre à un reclone doit aller dans un fichier versionné. Pas dans `~/claude/projects/`.



L'arbre de décision. Trois des quatre feuilles sont versionnées dans Git ; une seule est locale et éphémère.

Les quatre lieux

Lieu	Versionné Git ?	Scope	Cas d'usage
CLAUDE.md § Lessons learned	✅ Oui	Transversal projet	Pièges non-évidents, conventions implicites, décisions d'architecture, workarounds de versions de lib. Voir Chapitre 08.
.agents/skills/<domaine>/	✅ Oui	Connaissance métier spécifique	Règles canoniques avec exemples Incorrect → Correct. Chargées par les agents qui en ont besoin. Voir Chapitre 06.
.claude/agent-memory/<agent>/	✅ Oui	Par-agent : feedback & état projet	« Ne re-flagge pas », exceptions validées, références dont l'agent a spécifiquement besoin. Quatre sous-types : user_*, feedback_*, project_*, reference_*.
~/ .claude/projects/<projet>/memory/	❌ Non (local)	Cette machine, volatile	Préférences perso, contexte de session, raccourcis. Tout ce qui doit survivre à un reclone doit aller ailleurs.

Règles de décision

1. ≥ 2 agents bénéficieraient de cette info → CLAUDE.md § Lessons learned.
2. 1 agent, et le sujet est un domaine (Postgres, React, dates, commits) → skill sous .agents/skills/.
3. 1 agent, et l'info est purement « ne re-flagge pas » → .claude/agent-memory/<agent>/.
4. Aucun des trois ci-dessus (purement ma préférence, cette machine) → ~/ .claude/projects/.

Exemples concrets

Exemple 1. Tu as décidé d'utiliser date-fns plutôt que moment.js sur tout le projet. Plusieurs futurs commits y feront référence. Plusieurs agents (code-auditor, qa-tester) doivent le savoir. → CLAUDE.md § Lessons learned, avec le SHA du commit.

Exemple 2. Ton code-auditor flagge sans cesse les any dans les formatters Recharts, alors que c'est une convention projet documentée. → .claude/agent-memory/code-auditor/feedback_recharts_any_ok.md. L'agent charge sa mémoire et arrête de re-flagger.

Exemple 3. Tu veux un catalogue versionné de règles de versioning d'API REST avec exemples. → .agents/skills/api-versioning/ avec une règle par fichier sous references/.

Exemple 4. Tu préfères des réponses concises de Claude. C'est une préférence perso, cette machine uniquement. → ~/ .claude/projects/<projet>/memory/user_communication.md.

Anti-patterns

NE FAIS PAS ÇA

- **Dupliquer la même info dans deux lieux.** Au prochain update, l'un dérive. Choisis un seul lieu et fais pointer l'autre vers.
- **Mettre une convention projet critique dans `~/ .claude/projects/`.** C'est local. Ça sera perdu. Tu re-découvriras le bug à la dure après un reclone.
- **Mettre une formule dans une skill *et* dans le system prompt d'un agent.** La skill gagne. Toujours. Le prompt de l'agent doit référencer la skill, pas la dupliquer.
- **Traiter `CLAUDE.md` comme un dépôt.** S'il dépasse 300 lignes, splitte en imports thématiques (`@.claude/CONVENTIONS.md` , `@.claude/STACK.md` , etc.). Voir Chapitre 13.

MCP — Connecter à l'extérieur

Le **Model Context Protocol** donne à Claude accès à des outils externes (DB, navigateur, filesystem, APIs). Configuration dans `.mcp.json` à la racine :

```
JSON
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["-y", "@playwright/mcp@latest", "--browser=chromium", "--isolated"]
    },
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "./data"]
    },
    "tavily": {
      "type": "http",
      "url": "https://mcp.tavily.com/mcp/",
      "headers": { "X-API-Key": "${TAVILY_API_KEY}" }
    }
  }
}
```

Quels MCPs installer en premier

MCP	Pourquoi	Priorité
Playwright	Tester l'UI dans un navigateur. « Ouvre le dashboard et vérifie X. »	essentiel pour les web apps
filesystem	Accès en lecture à un dossier précis (données CSV, exports)	au cas par cas
Tavily	Recherche web premium (mieux que WebSearch par défaut)	si recherche fréquente
GitHub	Lire PRs, issues, comments sans quitter Claude	si workflows équipe
DB (Postgres, Supabase...)	Lire le schéma DB sans copy-paste — lock read-only	si la DB est centrale

SÉCURITÉ MCP

Un MCP qui peut écrire en DB prod = catastrophe potentielle. Lock **read-only** partout où possible. Les écritures passent par une migration explicite que tu reviewes à la main.

Le lock pattern read-only (nouveau en V2)

Pour tout MCP qui touche un système où des mutations sont possibles (DB, repo GitHub, filesystem), épingle la config en **read-only** via deux couches :

- Dans `.mcp.json` (committé), passe le flag read-only au serveur MCP lui-même.
- Dans `.claude/settings.local.json` (gitignored, local), restreins la liste des outils MCP aux opérations read-only.

Deux couches redondantes — si quelqu'un édite l'une, l'autre tient. Une seule couche échoue silencieusement. Exemple pour un MCP Postgres :


```
JSON
// .mcp.json
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres",
        "postgresql://readonly_user:***@host:5432/db",
        "--read-only"]
    }
  }
}

// .claude/settings.local.json (gitignored)
{
  "permissions": {
    "allow": [
      "mcp__postgres__query",
      "mcp__postgres__list_tables",
      "mcp__postgres__describe_table"
    ],
    "deny": [
      "mcp__postgres__execute"
    ]
  }
}
```

Pour le DDL ou les mutations, Claude rédige du SQL ; tu le colles dans ton vrai SQL editor. Lent par design. Le genre de lenteur qui prévient les accidents.

CHAPITRE 13 AVANCÉ

Maintenance long-terme

Ton setup est vivant. `CLAUDE.md` vieillit, les agents s'endorment, de nouveaux MCPs apparaissent. Voici la maintenance qui le garde sain.

Mémoire persistante des agents

Par défaut, un sub-agent oublie tout entre invocations. Pour les agents critiques, tu peux activer une mémoire trans-sessions versionnée dans Git :

1. Crée le dossier mémoire selon le scope souhaité (cf. table ci-dessous).
2. Ajoute `memory: project` (ou `user`, ou `local`) au frontmatter de l'agent.
3. Claude Code injecte les instructions de read/write et active les outils Read/Write/Edit pour l'agent.

Scope	Emplacement	Quand
<code>user</code>	<code>~/.claude/agent-memory/<nom>/</code>	Connaissance cross-projet (préférences perso, patterns réutilisables)
<code>project</code>	<code>.claude/agent-memory/<nom>/</code>	Connaissance projet, versionnée dans git, partageable en équipe (défaut recommandé)
<code>local</code>	<code>.claude/agent-memory-local/<nom>/</code>	Connaissance projet mais sensible / perso, gitignored

Splitter `CLAUDE.md` quand il grossit

Quand `CLAUDE.md` dépasse ~100 lignes, splitte en fichiers thématiques importés via `@` :

MARKDOWN

```
@AGENTS.md
@.claude/STACK.md
@.claude/CONVENTIONS.md
@.claude/WORKFLOW.md

# <Projet> – hub de configuration

| Tu cherches... | Va dans |
|---|---|
| Commandes + stack + gotchas | `@.claude/STACK.md` |
| Conventions, règles dev, workflow Git | `@.claude/CONVENTIONS.md` |
| Agents, slash commands, mode Coach | `@.claude/WORKFLOW.md` |
```

CLAUDE.md devient un hub qui pointe ; les fichiers thématiques restent < 80 lignes chacun, lisibles d'un coup d'œil.

Partager le setup en équipe

Tout `.claude/` versionné dans Git = chaque membre de l'équipe hérite automatiquement. Pour aller plus loin, packager en plugin (`.claude-plugin/<nom>/` avec `plugin.json`, un template CLAUDE.md, des hooks, agents, skills). Un plugin se partage comme une dépendance. Utile quand plusieurs projets internes partagent la même base.

La checklist `/audit-claude-setup` (étendue en V2)

Tous les ~30 jours, ou avant une release, lance `/audit-claude-setup`. La slash command complète vit en Annexe E ; voici la checklist qu'elle implémente :

1. **Inventaire.** Compte agents, slash commands, hooks, skills, mémoires actives, serveurs MCP. Compare à ton index `COMMANDS.md` — quelque chose non documenté ?
2. **Agents dormants.** Un agent qui n'apparaît jamais dans ton activity log est soit redondant, soit avec une `description` mal calibrée. Décide.
3. **Références cassées.** Pour chaque fichier mentionné dans `CLAUDE.md`, vérifie qu'il existe.
4. **Permissions inutilisées.** Une entrée `Bash(*)` jamais invoquée est du bruit — restreins-la.
5. **Mémoires agent vides.** Un agent `memory: project` dont le `MEMORY.md` est vide depuis 30+ jours ne capitalise rien. Sous-utilisé ou critères mal calibrés.
6. **Hooks lents.** Un hook qui prend > 500 ms ralentit chaque session. Profile, fix, ou passe en async.
7. **Modèles désalignés.** `opus` pour des agents triviaux gaspille des tokens ; `haiku` pour des complexes donne de mauvais résultats.
8. **Skills orphelines.** Une skill qu'aucun agent ne référence est du poids mort. Branche-la ou supprime-la. (nouveau en V2)
9. **Duplication skill ↔ prompt d'agent.** Si > 50 % du contenu d'une skill est aussi dans le prompt d'un agent, consolide. La skill gagne. (nouveau en V2)

ANNEXES

Templates & références.

Fichiers copy-paste-ready et un glossaire. Versions complètes dans le repo GitHub.

ANNEXE A

CLAUDE.md universel

MARKDOWN

<Nom du projet>

Description en une phrase.

Stack

- Langage / Framework / DB / Tests / Déploiement

Commandes utiles

- ``<dev>``, ``<verify>``, ``<test>``, ``<build>``

Conventions

- Nommage, structure, comportement

Gotchas

- Pièges stack et projet connus

Workflow Git

- Branche, format de commit, PR / push direct

Règles impératives

- **Jamais** X sans confirmation
- **Toujours** Y avant Z

Mode Coach (cf. Chapitre 09)

| Déclencheur | Action |

|---|---|

| ≥ 3 fichiers touchés OU module critique | /new-feature <slug> |

| Commit significatif (feat/fix/refactor) | /extract-lesson |

| Avant tout push | /ship "<message>" |

Lessons learned (cf. Chapitre 08)

> Section maintenue par /extract-lesson. Plus récent en haut.

ANNEXE B

.claude/settings.json complet

JSON

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Read", "Grep", "Glob",
      "Bash(git status:*)", "Bash(git diff:*)", "Bash(git log:*)",
      "Bash(git show:*)", "Bash(git branch:*)", "Bash(git fetch:*)",
      "Bash(ls:*)", "Bash(cat:*)", "Bash(find:*)",
      "Bash(head:*)", "Bash(tail:*)", "Bash(wc:*)"
    ],
    "deny": [
      "Bash(rm -rf*)", "Bash(rm)", "Bash(sudo*)",
      "Bash(curl* | bash*)"
    ]
  },
  "hooks": {
    "SessionStart": [
      { "matcher": "*", "hooks": [
        { "type": "command", "command": ".claude/hooks/session-start.sh" }
      ]
    },
    "PreToolUse": [
      { "matcher": "Edit|Write|Bash", "hooks": [
        { "type": "command", "command": ".claude/hooks/pre-tool-guard.sh" }
      ]
    },
    "PostToolUse": [
      { "matcher": "Edit|Write", "hooks": [
        { "type": "command", "command": ".claude/hooks/post-edit-format.sh" },
        { "type": "command", "command": ".claude/hooks/activity-log.sh", "async": true }
      ]
    },
    { "matcher": "Bash", "hooks": [
      { "type": "command", "command": ".claude/hooks/activity-log.sh", "async": true },
      { "type": "command", "command": ".claude/hooks/extract-lesson.sh" }
    ]
  },
    "Stop": [
      { "matcher": "*", "hooks": [
        { "type": "command", "command": ".claude/hooks/coach-suggest.sh" }
      ]
    }
  ]
}
```

ANNEXE C

Les six hooks (condensés)

Hook	Event	Rôle	Fichier complet
pre-tool-guard.sh	PreToolUse	Bloque les écritures et push dangereux	templates/.claude/hooks/pre-tool-guard.sh
post-edit-format.sh	PostToolUse	Formate les fichiers édités	templates/.claude/hooks/post-edit-format.sh
activity-log.sh	PostToolUse (async)	Log léger des appels d'outils	templates/.claude/hooks/activity-log.sh
session-start.sh	SessionStart	Récap 1 ligne de l'état git	templates/.claude/hooks/session-start.sh
coach-suggest.sh	Stop	Suggestions fin de session	templates/.claude/hooks/coach-suggest.sh
extract-lesson.sh NEW V2	PostToolUse	Suggère /extract-lesson après un commit significatif	templates/.claude/hooks/extract-lesson.sh

ANNEXE D

Quatre templates d'agents

Agent	Modèle	Pour
code-auditor	opus	Code mort, duplication, complexité, anti-patterns
security-auditor	opus	Secrets, XSS, injections, OWASP Top 10
plan-reviewer NEW V2	opus	Relit PLAN_<slug>.md avant code (mode staff-engineer). Étape 1.5 de /new-feature
doc-writer	sonnet	READMEs, ADRs, sections de changelog

System prompts complets dans templates/.claude/agents/ sur GitHub.

ANNEXE E

Six slash commands

Commande	Rôle
/ship "<msg>"	verify (tests + typecheck) + commit + push en 1 ligne
/audit-quick	code-auditor + security-auditor en parallèle ; rapport synthétisé
/standup	Récap quotidien : commits 24h + WIP + focus suggéré
/coach / /coach-mute / /coach-on	Toggle du mode Coach et recommandation active
/new-feature <slug> NEW V2	Pipeline Plan → plan-review → code → verify → audits
/extract-lesson [sha] NEW V2	Capture 1–3 leçons d'un commit vers CLAUDE.md

Glossaire

Agent (sub-agent)

Un prompt spécialisé + ensemble d'outils + choix de modèle, défini dans `.claude/agents/<nom>.md`, à qui Claude peut déléguer une tâche. Tourne dans son propre contexte.

\$ARGUMENTS

Placeholder pour les arguments passés à une slash command. Exemple : `/ship "fix: typo"` met `$ARGUMENTS` à `"fix: typo"`.

Mode Coach

Système à trois couches (règles dans `CLAUDE.md`, hook Stop avec suggestions, `/coach` manuel) qui te nudge vers la bonne commande/agent au bon moment.

Frontmatter

Bloc YAML en tête d'un fichier markdown (entre deux lignes `---`) qui contient les métadonnées. Utilisé dans les agents, slash commands, et références de skills.

Hook

Script shell que Claude Code lance à l'un des quatre événements (`SessionStart`, `PreToolUse`, `PostToolUse`, `Stop`) sans impliquer le LLM. Coût en tokens quasi-nul.

Least privilege (moindre privilège)

Pattern qui consiste à ne donner à chaque agent que les outils dont il a besoin (Read-only pour un auditeur, Read+Write pour un générateur). Réduit la surface d'attaque et les erreurs.

MCP

Model Context Protocol — le standard ouvert pour connecter Claude à des outils externes (DB, GitHub, navigateurs). Configuré via `.mcp.json`.

Matrice mémoire

La règle de décision pour savoir où stocker une info donnée : `CLAUDE.md`, une skill, une mémoire d'agent, ou la mémoire utilisateur locale. Cf. Chapitre 11.

Page-break-inside: avoid

Règle CSS qui dit au renderer PDF de garder un bloc sur une seule page plutôt que de le couper. Critique pour les blocs de code et les tables.

Permission denylist

Section `permissions.deny` de `settings.json` qui bloque des commandes précises (ex : `sudo`, `rm -rf /`) quel que soit le match avec l'`allowlist`.

Plan-first development

Pattern de workflow où l'utilisateur écrit un document `PLAN_<slug>.md` avant que la moindre ligne de code soit touchée. Relu par une 2e instance Claude (plan-reviewer) avant que l'implémentation démarre. Cf. Chapitre 07.

Skill

Dossier versionné de connaissances métier sous `.agents/skills/<nom>/` avec un point d'entrée `SKILL.md` et des fichiers de règles dans `references/`. Chargée à la demande. Cf. Chapitre 06.

Slash command

Template de prompt réutilisable défini dans `.claude/commands/<nom>.md`, invoqué en tapant `/nom` dans Claude Code.

Worktree

Un 2e répertoire de travail pour le même repo git, sur une branche différente. Te laisse lancer plusieurs sessions Claude Code en parallèle. Cf. Chapitre 10.

ANNEXE G

Quick start 30 minutes

#	Étape	Temps
1	Installer Claude Code (<code>brew install ...</code> ou plateforme-spécifique)	5 min
2	Créer <code>CLAUDE.md</code> à la racine (stack, conventions, règles d'or)	10 min
3	Créer <code>.claude/settings.json</code> avec allowlist + denylist + 3 hooks	5 min
4	Ajouter les 3 scripts hooks (<code>pre-tool-guard.sh</code> , <code>post-edit-format.sh</code> , <code>session-start.sh</code>) et <code>chmod +x</code>	3 min
5	Ajouter <code>.claude/logs/</code> et <code>.claude/settings.local.json</code> au <code>.gitignore</code>	1 min
6	Redémarrer Claude Code ; vérifier que la bannière SessionStart apparaît	1 min

Après ça, les 30 minutes suivantes te donnent les sub-agents (Chapitre 05). L'heure d'après te donne les slash commands (Chapitre 04). N'essaie pas de tout faire d'un coup.

ANNEXE H

Deux études de cas

Deux histoires anonymisées — un dev, une knowledge worker — qui appliquent le setup. Pas de nom de produit, pas de spécificités de domaine. Les enseignements se généralisent.

Cas 1 — Dev SaaS solo (6 mois)

Environ 30 k LOC de TypeScript, déploiements quotidiens, ~40 commits par semaine. Stack web moderne avec une DB transactionnelle et un dashboard analytique. Dev solo avec un an d'expérience de code.

« Je passais 15 à 20 minutes par session à ré-expliquer le contexte. Huit builds CI cassés en deux semaines à cause d'un `any` implicite. Aucune trace écrite des décisions d'archi — toutes les deux semaines je re-débattais d'un truc que j'avais déjà tranché. »

Ce qui a été configuré

- Un hub `CLAUDE.md` + trois imports thématiques (`STACK.md` , `CONVENTIONS.md` , `WORKFLOW.md`)
- 17 sub-agents (10 techniques, 5 marketing, 2 méta)
- 6 hooks (les 5 standards du livre + le hook extract-lesson)
- 14 slash commands dont `/new-feature` avec plan-reviewer à l'étape 1.5
- 5 skills, une par domaine technique (SQL best practices, gestion des dates, conventions de commits, etc.)

Résultats à 6 mois

Métrique	Avant	Après
Temps de ré-explication de contexte par session	15–20 min	< 1 min
Builds CI cassés par semaine	2–3	0–1
Fichiers de doc projet	3 READMEs épars	14 fichiers <code>.md</code> structurés
Sub-agents invoqués proactivement	n/a	8/12 actifs (4 dormants, supprimés au prochain audit)
Leçons capturées dans Git	0	23 entrées dans § Lessons learned

Ce qui a marché

- Plan-first via `/new-feature`. Zéro refactor majeur en 6 mois — rattrapé au planning, pas après le code.
- Skills > inline prompts. Dérive éliminée après le 1er audit mensuel qui a révélé deux skills dupliquées dans des prompts d'agents.
- La matrice mémoire. Plus de doute sur « où écrire ça ».

Ce qui n'a PAS marché

- 4 sub-agents jamais invoqués. Soit redondants, soit avec une `description` mal calibrée. 2 fusionnés, 2 supprimés.
- Worktrees parallèles abandonnés après un essai — le laptop n'avait pas la RAM pour 3 dev servers concurrents.
- Un hook TDD-guard parfois trop strict sur les fichiers UI cosmétiques. Mitigation : une entrée ciblée dans agent-memory.

Citation de clôture

« Le setup ne te rend pas plus rapide — il te rend reproductible. C'est l'opposé du glamour, mais c'est ce qui m'a permis de tenir 6 mois sans abandonner. »

Cas 2 — Knowledge worker / chercheuse indépendante (3 mois)

Une rédactrice technique freelance et analyste de recherche qui utilise Claude Code comme outil quotidien d'écriture et d'analyse. Pas de code en production, mais usage intensif de longs documents, vérification de faces, prise de notes structurée. Travaille sur deux repos — un pour les livrables client, un pour une base de connaissances perso.

« J'utilisais Claude dans un onglet de navigateur depuis un an. Chaque conversation commençait par coller mon style guide, mes sources, le contexte de mon projet. Je perdais deux heures par semaine rien que sur ce rituel. J'ai essayé Claude Code sur recommandation d'un ami ; avec le setup de ce manuel, le rituel a disparu. »

Ce qui a été configuré

- Un `CLAUDE.md` à la racine du repo de base de connaissances avec : style guide, sources récurrentes (et format de citation), règles de ton de voix, expressions à proscrire
- 3 sub-agents : `fact-checker` (vérifie les affirmations contre les sources avant rédaction), `tone-reviewer` (flagge la dérive du style guide), `structure-reviewer` (vérifie hiérarchie des titres et distribution de longueurs)
- 4 hooks : `session-start.sh` montre les « 3 derniers documents touchés », `pre-tool-guard.sh` bloque les écritures hors de `drafts/` et `published/`, `post-edit-format.sh` lance `prettier` sur le markdown, `extract-lesson.sh` nudge pour capturer les décisions éditoriales
- 3 slash commands : `/draft <sujet>` (recherche + outline), `/polish` (les 3 reviewers en parallèle), `/cite <url>` (fetch + format)

- 1 skill : `citation-styles` avec règles pour formats académique, journalistique et blog — élimine la ré-explication à chaque essai

Résultats à 3 mois

Métrique	Avant	Après
Temps de rituel en début de session	~10 min	0 min
Articles publiés par mois	4	7
Dérive du style guide rattrapée avant publication	par l'éditeur, après coup	par <code>tone-reviewer</code> , en amont
Heures économisées par semaine (auto-déclarées)	—	5–7

Citation de clôture

« Je ne suis pas développeuse, et la plupart du contenu sur Claude Code suppose que tu l'es. Ce manuel a expliqué le même setup d'une manière qui colle à mon workflow. Les patterns — planifier avant, capturer les leçons après, une skill par domaine — se traduisent un pour un en écriture. »

Ce que les deux cas partagent

Même résultat, domaines différents : **le rituel a disparu, les standards ont tenu, le travail s'est composé.** C'est toute la promesse du setup.

ANNEXE I

Cheat sheet

Un foldout deux pages. Imprime en A3 (une face par page) ou garde numérique. Tout ce dont tu as besoin sans ouvrir le livre.

Hooks · les 6

Hook	Event	Rôle
<code>session-start.sh</code>	SessionStart	Récap 1 ligne de l'état git, branche, derniers commits
<code>pre-tool-guard.sh</code>	PreToolUse	Bloque écritures <code>.env</code> , <code>rm -rf /</code> , force push
<code>post-edit-format.sh</code>	PostToolUse	Formate les fichiers édités (prettier, ruff, etc.)
<code>activity-log.sh</code>	PostToolUse async	Log léger des appels d'outils pour audit
<code>coach-suggest.sh</code>	Stop	Suggestions de fin de session selon ce qui a été touché
<code>extract-lesson.sh</code>	PostToolUse	Après commit significatif, nudge <code>/extract-lesson</code>

Slash commands · les 8

Commande	Rôle
<code>/ship "<msg>"</code>	verify + commit + push en 1 ligne
<code>/audit-quick</code>	code-auditor + security-auditor en parallèle
<code>/standup</code>	Récap quotidien : commits 24h + WIP + focus suggéré

<code>/coach / /coach-mute / /coach-on</code>	Mode Coach call actif / toggle mute
<code>/new-feature <slug></code>	Pipeline Plan → plan-review → code → verify → audits
<code>/extract-lesson [sha]</code>	Capture 1-3 leçons d'un commit vers CLAUDE.md

Templates d'agents · les 4

Agent	Modèle	Pour
<code>code-auditor</code>	opus	Code mort, duplication, complexité, anti-patterns
<code>security-auditor</code>	opus	Secrets, XSS, injections, OWASP Top 10
<code>plan-reviewer</code>	opus	Relit <code>PLAN_<slug>.md</code> avant code (mode staff-engineer)
<code>doc-writer</code>	sonnet	READMEs, ADRs, sections de changelog

Mémoire · où écrire quoi

Lieu	Versionné ?	À utiliser pour
<code>CLAUDE.md § Lessons</code>	✓	≥ 2 agents en ont besoin · transversal
<code>.agents/skills/<domaine>/</code>	✓	Règles de domaine · structurées (Incorrect → Correct)
<code>.claude/agent-memory/<agent>/</code>	✓	« Ne re-flagge pas » · par-agent
<code>~/.claude/projects/.../memory/</code>	✗	Perso · cette machine uniquement · éphémère

La règle de décision, en une phrase

Slash command si c'est un verbe. **Sub-agent** si c'est un nom avec un boulot. **Skill** si c'est un corpus de règles. **Hook** si c'est un réflexe.

Les règles d'or

1. `pnpm verify` (ou équivalent) doit être vert avant chaque push. Husky pre-push si tu es du genre à oublier.
2. Plan-first quand le chantier touche ≥ 3 fichiers OU un module critique OU une migration DB.
3. Après un commit significatif, décide : capture une leçon, ou laisse le hook silencieux.
4. Chaque info vit dans exactement UN des quatre lieux mémoire. Pas de doublon.
5. Skill > prompt d'agent. Si une règle vit dans les deux, l'agent perd ; la skill gagne.
6. Lance `/audit-claude-setup` une fois par mois. Skills orphelines, agents dormants, refs cassées.